

**CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA “PAULA SOUZA”
FACULDADE DE TECNOLOGIA DE TAQUARITINGA
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS**

COBECO - COTAÇÃO DE BENS DE CONSUMO

**LEONARDO DAVID SILVA SETTI
FABRICIO DE LIMA CABRAL
NICHOLAS ANTHONY FERREIRA BRAZ
EDUARDO PEREIRA PASSARELLI
NICOLAS HENRIQUE MALLOUK**

PROF.(A) ORIENTADOR(A): THAIS CRISTINA CASAGRANDE

TAQUARITINGA - S.P.

2026

LEONARDO DAVID SILVA SETTI
FABRICIO DE LIMA CABRAL
NICHOLAS ANTHONY FERREIRA BRAZ
EDUARDO PEREIRA PASSARELLI
NICOLAS HENRIQUE MALLOUK

COBECO - COTAÇÃO DE BENS DE CONSUMO

Monografia apresentada à Faculdade de
Tecnologia de Taquaritinga, como parte
dos requisitos para a obtenção do título
de Tecnólogo em Análise e
Desenvolvimento de Sistemas.

Orientadora: Profa. Thais Cristina
Casagrande

TAQUARITINGA - S.P.

2026

SUMÁRIO

1 OBJETIVO DO PROJETO

2 ESCOPO

3 REQUISITOS

3.1 Funcionais

3.1.1 Autenticação e controle de acesso

3.1.2 Gestão de listas e produtos

3.1.3 Cotação por paridade

3.1.4 Histórico e persistência

3.1.5 Validação e exceções

3.2 Não-Funcionais

3.2.1 Arquitetura e infraestrutura

3.2.2 Testes e qualidade

3.2.3 Experiência e acessibilidade

3.2.4 Segurança, desempenho e stack

3.3 Desejáveis

4 TECNOLOGIAS, MODELAGEM E PROTÓTIPOS

4.1 Tecnologias adotadas

4.2 Modelagem, protótipos e qualidade

5 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 OBJETIVO DO PROJETO

O COBECO (Cotação de Bens de Consumo) é uma aplicação web destinada a usuários autenticados que desejam organizar listas e comparar fornecedores de forma transparente. O núcleo do MVP é o motor de paridade: para cada lista, o sistema identifica os itens disponíveis em cada fornecedor, agrupa fornecedores com o mesmo perfil de cobertura, calcula os totais e evidencia produtos ausentes e o orçamento consolidado. Os preços do MVP provêm de catálogo controlado e persistido no PostgreSQL, o que assegura uma demonstração determinística e independente de serviços externos. Listas e cotações permanecem disponíveis no histórico do usuário, com autenticação, validação e tratamento de exceções.

2 ESCOPO

O escopo inclui cadastro, login, logout e recuperação de senha; gestão completa de listas categorizadas e seus itens; catálogo normalizado de categorias, produtos, fornecedores e ofertas; seleção dos fornecedores; cotação de lista; agrupamento por perfil idêntico de cobertura; cálculo do percentual, dos itens ausentes, dos totais por fornecedor e do menor custo consolidado; painel comparativo; persistência e reabertura do histórico; exportação CSV e impressão; contrato OpenAPI; e execução em contêineres separados para interface, API e banco. Não integram o MVP compras, pagamentos, garantia de preço em tempo real, aplicativo móvel nativo, autenticação social ou dependência obrigatória de APIs externas. Integrações já desenvolvidas são preservadas, mas ficam desativadas por padrão como evolução pós-MVP.

3 REQUISITOS

Os requisitos do COBECO foram levantados a partir dos objetivos e do escopo previamente definidos e encontram-se organizados em três categorias: requisitos funcionais, que descrevem as funcionalidades que o sistema deve oferecer; requisitos não funcionais, que expressam restrições e atributos de qualidade da solução; e requisitos desejáveis, que não integram a versão mínima viável, mas agregam valor ao produto e poderão ser incorporados em evoluções futuras.

3.1 Funcionais

Os requisitos funcionais correspondem às ações que o sistema deve executar para atender às necessidades dos usuários. Encontram-se agrupados conforme os módulos da aplicação, sendo apresentados nos Quadros 1 a 5.

3.1.1 Autenticação e controle de acesso

Quadro 1 – Requisitos funcionais da área pública e do controle de acesso

Código	Descrição do requisito
RF01	Permitir cadastro com nome, e-mail único, senha e consentimento.
RF02	Autenticar por e-mail e senha e direcionar à área logada.
RF03	Encerrar a sessão com segurança.
RF04	Solicitar e concluir a redefinição de senha.
RF05	Proteger todas as rotas da plataforma.

Fonte: elaborado pelo autor (2026).

3.1.2 Gestão de listas e produtos

Quadro 2 – Requisitos funcionais de gestão de listas e produtos

Código	Descrição do requisito
RF06	Criar listas nomeadas vinculadas a uma categoria.
RF07	Incluir item com produto normalizado ou descrição livre, categoria e quantidade.
RF08	Editar o nome da lista e editar ou remover seus itens.
RF09	Duplicar uma lista e seus itens.
RF10	Excluir uma lista após confirmação.
RF11	Exibir as listas do usuário, seus itens e datas.

Fonte: elaborado pelo autor (2026).

3.1.3 Cotação por paridade

Quadro 3 – Requisitos funcionais de cotação e comparação de preços

Código	Descrição do requisito
RF12	Solicitar a cotação de todos os itens de uma lista.
RF13	Agrupar fornecedores com exatamente o mesmo conjunto de itens disponíveis e exibir o percentual de cobertura.
RF14	Calcular, por grupo, totais individuais e o menor custo consolidado por item.
RF15	Destacar visualmente o melhor grupo calculado.
RF16	Listar os produtos ausentes em cada grupo.
RF17	Permitir incluir ou excluir fornecedores da comparação.
RF18	Apresentar painel com fornecedores, cobertura, preços, ausências e orçamento consolidado.

Fonte: elaborado pelo autor (2026).

3.1.4 Histórico e persistência

Quadro 4 – Requisitos funcionais de histórico e persistência de dados

Código	Descrição do requisito
RF19	Persistir cotações, data, grupos e resultado destacado.
RF20	Visualizar e reabrir cotações anteriores.
RF21	Excluir registros do histórico pertencentes ao usuário.

Fonte: elaborado pelo autor (2026).

3.1.5 Validação e exceções

Quadro 5 – Requisitos funcionais de tratamento de exceções

Código	Descrição do requisito
RF22	Informar quando nenhum fornecedor ou oferta válida for encontrado.
RF23	Sinalizar produto não mapeado ou indisponível.
RF24	Validar nome, quantidade, categoria e seleção antes da cotação.

Fonte: elaborado pelo autor (2026).

3.2 Não-Funcionais

Os requisitos não funcionais estabelecem os atributos de qualidade e as restrições sob as quais o sistema deve operar, abrangendo segurança, desempenho, usabilidade e características de arquitetura, conforme apresentado nos Quadros 6 a 9.

3.2.1 Arquitetura e infraestrutura

Quadro 6 – Requisitos não funcionais de segurança e privacidade

Código	Descrição do requisito
RNF01	Separar domínio puro, serviços, adapters de persistência e framework web.
RNF02	Manter API REST regida por contrato OpenAPI e adotar API-First nas novas rotas.
RNF03	Usar PostgreSQL com transações ACID e migrations versionadas.
RNF04	Disponibilizar frontend, API e PostgreSQL como três serviços Docker.

Fonte: elaborado pelo autor (2026).

3.2.2 Testes e qualidade

Quadro 7 – Requisitos não funcionais de desempenho e disponibilidade

Código	Descrição do requisito
RNF05	Descrever regras críticas em Gherkin e executá-las com Cucumber.
RNF06	Cobrir domínio e serviços com testes Vitest.
RNF07	Cobrir cadastro, login, lista e cotação com Playwright.
RNF08	Executar lint, testes e build no GitHub Actions.

Fonte: elaborado pelo autor (2026).

3.2.3 Experiência e acessibilidade

Quadro 8 – Requisitos não funcionais de usabilidade e interface

Código	Descrição do requisito
RNF09	Otimizar a interface para desktop a partir de 1024 × 768.
RNF10	Exibir estados de carregamento e erros compreensíveis.
RNF11	Garantir navegação por teclado, foco, contraste, semântica e rótulos.
RNF12	Destacar o melhor orçamento sem depender somente de cor.

Fonte: elaborado pelo autor (2026).

3.2.4 Segurança, desempenho e stack

Quadro 9 – Requisitos não funcionais de arquitetura e manutenção

Código	Descrição do requisito
RNF13	Armazenar senhas somente com hash Argon2.
RNF14	Usar HTTPS no ambiente de produção.
RNF15	Expirar a sessão após 30 minutos sem renovação ou atividade.
RNF16	Indexar chaves usadas no catálogo, histórico e relacionamentos.
RNF17	Servir a UI estática independentemente da disponibilidade da API.
RNF18	Utilizar tecnologias FOSS ou de uso gratuito.
RNF19	Implementar o backend em Node.js, TypeScript e Express.
RNF20	Implementar o frontend em React 18 e TypeScript, sem jQuery.
RNF21	Usar PostgreSQL 16 e Prisma ORM 5.
RNF22	Versionar no GitHub e adotar GitHub Flow.

Fonte: elaborado pelo autor (2026).

3.3 Desejáveis

Os requisitos desejáveis não são obrigatórios para a entrega da versão mínima viável do sistema, porém agregam valor ao produto e poderão ser incorporados em versões posteriores, conforme a disponibilidade de tempo e de recursos da equipe. Tais requisitos são apresentados no Quadro 10.

Quadro 10 – Requisitos desejáveis

Código	Descrição do requisito
RD01	Autenticação por contas de terceiros.
RD02	Alerta de preço abaixo de limite definido.
RD03	Exportação avançada dos resultados em PDF ou planilha.
RD04	Gráficos de variação histórica de preços.
RD05	Ampliação do compartilhamento e colaboração em listas.
RD06	Novas categorias e painel administrativo do catálogo.

Fonte: elaborado pelo autor (2026).

4 TECNOLOGIAS, MODELAGEM E PROTÓTIPOS

A solução foi implementada como monorepo TypeScript. O domínio de disponibilidade, agrupamento e orçamento é independente de Express e Prisma. A API expõe contratos REST documentados em OpenAPI e persiste dados com Prisma/PostgreSQL. A interface React permite selecionar fornecedores e apresenta grupos, cobertura, ausências, totais individuais e destaque textual do resultado.

4.1 Tecnologias adotadas

Foram adotados Node.js 20, Express 4.22, TypeScript 5.9, Zod 3.25, JWT, Argon2 0.44, Prisma 5.22 e PostgreSQL 16 no backend; React 18.3, React Router 7.18, Vite 8.2 e Tailwind CSS 3.4 no frontend; Vitest 4.1, Cucumber 13.2 e Playwright 1.62 para qualidade; OpenAPI 3.0.3 para o contrato; e Docker Compose com Nginx para os três serviços. Todas as tecnologias são FOSS ou de uso gratuito.

4.2 Modelagem, protótipos e qualidade

O DER deriva do schema Prisma e inclui usuários, listas, itens, categorias, produtos, fornecedores, ofertas e cotações. O diagrama de casos de uso foi revisado para representar o catálogo persistido e o agrupamento por paridade. Os protótipos de baixa fidelidade estão nos arquivos Excalidraw e o protótipo de alta fidelidade é a interface React executável. O cenário A–H gera quatro grupos e é validado por teste unitário, integração HTTP e cenário Gherkin; o fluxo crítico foi validado no Chromium.

5 CONSIDERAÇÕES FINAIS

O projeto alcança o objetivo de transformar uma lista de bens de consumo em uma comparação clara de cobertura e custo. Ao agrupar fornecedores pelo conjunto real de itens disponíveis, o COBECO evita que percentuais iguais ocultem perfis distintos e torna explícitas as pendências de cada alternativa. O catálogo seedado reduz riscos externos, enquanto PostgreSQL, OpenAPI, contêineres e testes tornam a solução reproduzível. Como continuidade, recomenda-se ampliar o catálogo, criar sua administração, automatizar auditorias de acessibilidade e publicar a aplicação atrás de HTTPS.

REFERÊNCIAS

OPENAPI INITIATIVE. OpenAPI Specification 3.0.3. 2020.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation. 2026.

PRISMA DATA. Prisma ORM Documentation. 2026.

META PLATFORMS. React Documentation. 2026.

